

Orion Protocol Specification Version 0.1

Ronan Redwood

September 20 2026

1 Introduction

The Orion protocol is a lightweight, text-based application protocol designed for simple content retrieval over TCP. It emphasizes minimalism, clarity, and ease of implementation. Orion is intended as a simpler alternative to the Gemini protocol.

2 Conventions

All text in this specification uses UTF-8 encoding. Line endings **MUST** be CRLF. The key words **MUST**, **SHOULD**, and **MAY** are to be interpreted as described in RFC 2119.

3 URI Scheme

The Orion protocol uses the `orion://` URI scheme. The general form is:

`orion://host[:port]/path[?query] [#fragment]`

3.1 Host

The host component **MUST** be present and **MUST** be either a DNS name or an IPv4/IPv6 address.

3.2 Port

If no port is specified, the default port 4040 is assumed.

3.3 Path

The path component identifies the resource requested. If omitted, the path defaults to `/`.

3.4 Query and Fragment

The query and fragment components **MAY** be present. Their interpretation is left to server implementations and is opaque to the protocol itself.

4 Connection Model

Orion is a simple request/response protocol operating over TCP.

- Clients **MUST** send exactly one request per TCP connection.
- Servers **MUST** send exactly one response per TCP connection.
- After sending the response, the server **MUST** close the connection.

- TLS MAY be used, but is not required in Version 0.1.

5 Request Format

A request consists of a single line containing an absolute Orion URI, terminated by CRLF. No headers or body are permitted.

`<absolute-URI><CRLF>`

The request line MUST NOT exceed 1024 bytes. Requests exceeding this limit MUST be rejected with status code 40 (Temporary Failure).

6 Response Format

A response begins with a status code, a single space, and a meta field, terminated by CRLF. An optional body MAY follow.

`<status-code> <meta><CRLF>`
`<body>`

The meaning of the meta field depends on the status code class.

7 Status Codes

Orion defines the following status code classes:

7.1 2x Success

- 20 Success — The request was valid and the body contains the resource.

7.2 3x Redirect

- 30 Redirect — The meta field contains a new Orion URI.

7.3 4x Temporary Failure

- 40 Temporary Failure — The request could not be fulfilled; retry later.

7.4 5x Permanent Failure

- 51 Not Found — The requested resource does not exist.

8 Example

Client request:

`orion://example.com/\r\n`

Server response:

`20 text/orion\r\n`
`Welcome to Orion!`

9 Security Considerations

Orion Version 0.1 does not require TLS, but servers and clients MAY use it. Implementations MUST NOT execute active content, scripts, or embedded code. Cookies, compression, and persistent connections are intentionally omitted to reduce attack surface and simplify implementation.

10 OTXT

The Orion protocol uses the OTXT file type to host pages on Orion Net Version 0.1. OTXT is a plain UTF-8 text format that is entirely line-based. The syntax is inspired by Markdown but intentionally simplified.

10.1 Headings

Headings are created using leading # characters:

- # Heading — level 1 (largest)
- ## Heading — level 2 (medium)
- ### Heading — level 3 (smallest)

10.2 Links

Links are defined on their own line, starting with =>:

- => orion://host/path Link text

The first token after => is the URI. Any text following the URI on the same line is treated as the human-readable label.

10.3 Lists

List items begin with * or -:

- * Item text
- - Item text

10.4 Code Blocks

Preformatted text or code blocks are surrounded by lines containing three backticks:

```
'''
print("Hello from Orion")
'''
```

10.5 Paragraphs

Any line that does not begin with a special marker (#, =, *, -, or “”) is treated as a paragraph of plain text.